AIOPS-06: AI Integrations and Agentic Workflows

> **Series:** AIOPS — Dynatrace Intelligence | **Notebook:** 6 of 8 | **Created:** May 2026 | **Last Updated:** 10/12/2026

Overview

Dynatrace Intelligence is not a closed system. The same AI capabilities are available via Workflow tasks (for in-platform automation) and via the Dynatrace MCP server (for external IDEs, CLI agents, and bring-your-own AI orchestration).

This notebook covers three integration surfaces: AI tasks inside Workflows, the Dynatrace MCP server for IDE / agent integration, and the canonical workflow tutorials from the documentation.

Audience: Platform engineer building automation; SRE integrating Dynatrace into IDE and CLI workflows; AI / governance lead reviewing the integration surface.

Outcome: Working knowledge of where Dynatrace Intelligence connects out — and which patterns are settled vs. still emerging.

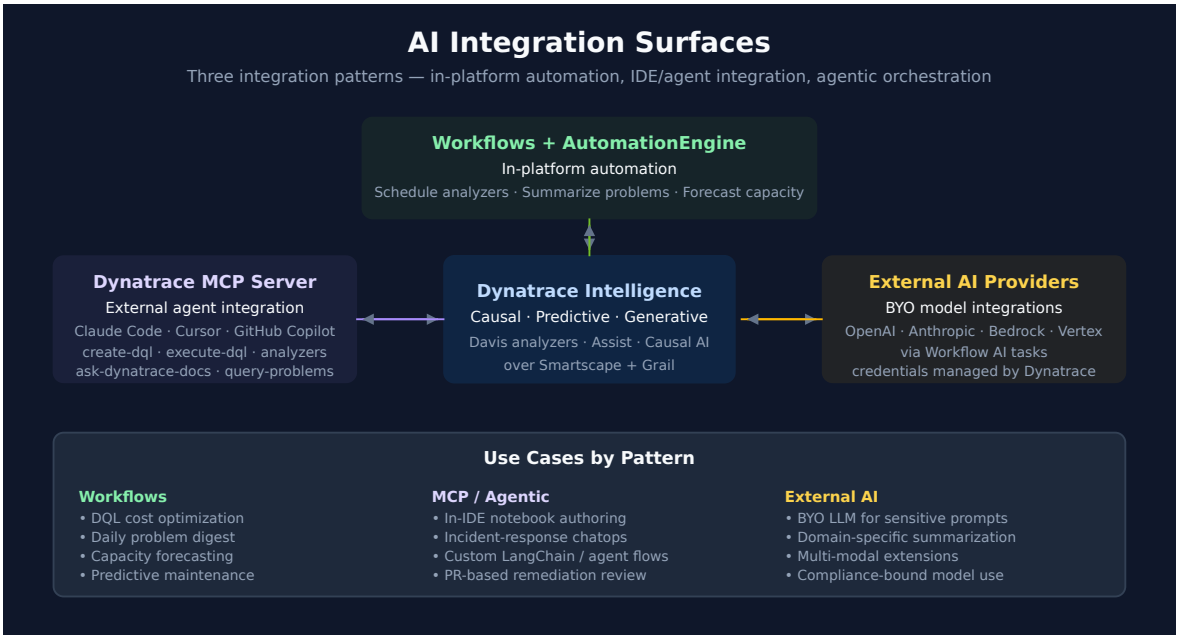


Table of Contents

1. Three Integration Surfaces
    - AI Observability: Monitoring Your Own GenAI Applications
2. Workflow Tutorial: Optimize DQL Cost
3. Workflow Tutorial: Summarize Open Problems
4. Workflow Tutorial: Forecast Resource Utilization
5. The Dynatrace MCP Server
6. Agentic Patterns: AutomationEngine and Approval-Based Remediation
7. Cross-Series Pointers

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3
Apps	Workflows app; Notebooks app
Permissions	`davis:analyzers:execute`, `storage:events:read`, `automation:workflows:run`, `automation:workflows:write` (§2–§4 build workflows)
For MCP integrations	Dynatrace MCP server installed in your IDE / CLI agent (Claude Code, Cursor, GitHub Copilot)
For external LLMs	Provider credentials (OpenAI / Anthropic / Bedrock / Vertex) configured as Dynatrace credentials

1. Three Integration Surfaces

Workflow AI tasks (in-platform). Dynatrace Workflows can call AI tasks: run an analyzer, generate a summary, route through an external LLM. Best for *scheduled* and *event-driven* automation that lives entirely inside the platform.

Dynatrace MCP server (platform → external agent). Exposes the platform's AI surface as MCP tools — `create-dql`, `execute-dql`, the analyzer family, `ask-dynatrace-docs`, `find-troubleshooting-guides`. Best when your team wants Dynatrace context inside Claude Code, Cursor, or GitHub Copilot.

AutomationEngine. The orchestration layer that wraps workflows for higher-level automation patterns — predictive maintenance, approval-based remediation, ChatOps integration.

These three are complementary, not exclusive. A mature observability practice uses all three for different jobs.

> **Adjacent capability — AI Observability (monitoring *your own* GenAI apps).** Distinct from the three Dynatrace-Intelligence integration surfaces above: **AI Observability** ingests GenAI / LLM telemetry from your *own* applications as OpenTelemetry spans. It reads Dynatrace's native `gen_ai.*` attributes — so native OTel, **OpenLLMetry**, and OneAgent's Python GenAI auto-instrumentation (Bedrock / OpenAI / Azure OpenAI / LangChain, OneAgent 1.339+) flow straight in. **OpenInference** (the Arize AI standard) uses its own `llm.*` attributes and **must be normalized to `gen_ai.*` first** — either with an OTel Collector `transform` processor or via Dynatrace OpenPipeline (see the OPIPE series). Prerequisites: a DPS license with Traces powered by Grail, OTLP ingestion enabled, and an API token with the `openTelemetryTrace.ingest` scope.
>
> **Sources:** Get started with OpenInference and AI Observability (DT docs), AI Observability (DT docs).

1a. AI Observability: monitoring your own GenAI applications

Where it surfaces

SaaS 1.344 (rollout from 07/29/2026) added two dedicated surfaces: a **Smartscape view** for GenAI topology, and a standalone **Evaluations** screen for prompt evaluations.

On a tenant still below 1.344, prompt evaluations surface inside the general app views — the underlying span data is identical either way.

The conversation-reconstruction problem

Each turn of a conversational GenAI application is a **separate HTTP request with its own trace ID**. Traces are therefore per-turn by construction, and nothing in the default span data stitches turn 1 to turn 7. End-to-end session analysis — cost per conversation, where a multi-turn agent went wrong, which sessions got a bad answer — needs a **shared identifier carried across turns**.

Key attributes

Attribute	Role
`gen_ai.conversation.id`	Groups every LLM span belonging to one browser session — the join key for reconstruction
`session.id`	Alias for the same value within AI Observability
`dt.rum.session.id`	The RUM session identifier, propagated by the RUM JavaScript in the `tracestate` header — links the conversation to the frontend session
`traceparent`	W3C header injected by RUM to link browser and backend spans (see **OTEL-04 § 4**)

Supporting attributes for cost and model analysis: `gen_ai.provider.name`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens`.

> **Attribute-existence note.** `gen_ai.provider.name`, `gen_ai.request.model`, `gen_ai.usage.input_tokens` and `gen_ai.usage.output_tokens` are present in the Dynatrace semantic dictionary (stability: *experimental*), as is `dt.rum.session.id` (stability: *stable*). **`gen_ai.conversation.id` and `session.id` are not** — they are application-set span attributes that you populate, per the implementation below, not platform-declared fields. That is expected for this pattern, but it means a typo in the attribute name fails silently: the query returns zero rows rather than an error. Confirm the attribute is arriving before you build on it.

Implementation shape

The identifier has to be created on the frontend and survive all the way to span export:

1. **Frontend** generates a UUID once per conversation and stores it in `sessionStorage`.
2. Every request to the backend **passes that UUID in the request body**.
3. The backend holds it in a **`ContextVar`** (or the equivalent request-scoped store for your runtime) so it is reachable from code that has no access to the request object.
4. A **span processor** stages the value under a private attribute as spans are created.
5. An **exporter wrapper** writes it to **`gen_ai.conversation.id`** immediately before export.

Steps 4 and 5 are the part people skip. Setting the attribute at span *creation* only works where you control the span; the processor-plus-exporter pair is what gets the identifier onto spans created by auto-instrumentation and SDK internals too.

Reconstructing a conversation

The queries below are the documented reconstruction pattern. Both queries were executed against a live tenant (07/30/2026, re-run 09/24/2026): they parse and run, but that tenant carries no GenAI spans at all, so field population is unconfirmed — see the note under each cell.

```
// Reconstruct one conversation, turn by turn.
// Source: Dynatrace docs: Conversation ID and agent session tracking (sample
use case) —
// adapted: sorts on start_time, not timestamp.
// Executed against a live tenant 07/30/2026: the query runs, but that tenant
has no
// GenAI telemetry, so it returned 0 rows. Shape confirmed; field population
is not.
// Replace the id with one you captured from the frontend's sessionStorage.
fetch spans, from:-24h
| filter gen_ai.conversation.id == "1f0c4d2e-9a3b-4c71-8e55-6d2a0b7f1c34"
| fields start_time,
         span.name,
         gen_ai.request.model,
         gen_ai.usage.input_tokens,
         gen_ai.usage.output_tokens,
         feedback.rating
| sort start_time asc
| limit 200
```

Reading the result: one row per LLM span, in conversation order — the turn sequence, which model served each turn, and the token cost of each. `feedback.rating` appears only if your application writes a user-feedback attribute onto the span; it is not a platform field, so expect nulls until you add it.

Verification status: syntax-verified **and executed** against a live tenant on 07/30/2026 — the query parses and runs. It returned **0 rows**, because that tenant ingests no GenAI telemetry. So the query *shape* is confirmed; *field population* is not, and cannot be from this environment. Note also that `gen_ai.conversation.id`, `session.id` and `feedback.rating` are **application-set attributes, not platform fields** — they exist only if your instrumentation writes them, they will not appear in the semantic dictionary, and a misspelling fails **silently** with zero rows rather than an error. Confirm the attributes are populated in your own environment before building on it.

Aggregating across conversations turns the same join key into a cost view:

```
// Token usage per conversation — which sessions drive spend.
// Uses the same gen_ai.conversation.id join key, aggregated.
// Executed against a live tenant 07/30/2026: the query runs, but that tenant
has no
// GenAI telemetry, so it returned 0 rows. Shape confirmed; field population
is not.
fetch spans, from:-24h
| filter isNotNull(gen_ai.conversation.id)
| summarize {
    turns = count(),
    input_tokens = sum(gen_ai.usage.input_tokens),
    output_tokens = sum(gen_ai.usage.output_tokens),
    models = collectDistinct(gen_ai.request.model)
  }, by: {gen_ai.conversation.id}
| fieldsAdd total_tokens = input_tokens + output_tokens
| sort total_tokens desc
| limit 50
```

Reading the result: the long-tail conversations at the top of this list are where token spend concentrates — usually a small number of sessions with many turns, or one model being called where a cheaper one would do. Cross-reference **FINOPS-01** for how that translates into DPS consumption.

Verification status: syntax-verified **and executed** 07/30/2026; returned 0 rows for the same reason as above — no GenAI telemetry on the validation tenant, not a query defect.

> **Sources:** Conversation ID and agent session tracking (DT docs) — *"A `SessionIdExporter` wrapper copies the staged value to `gen_ai.conversation.id`"*, Semantic Dictionary — `gen_ai` fields (DT docs). **Dictionary:** `gen_ai.provider.name`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`, `gen_ai.usage.output_tokens` (`experimental`), `dt.rum.session.id` (`stable`); no row for `gen_ai.conversation.id` or `session.id` under `filter contains(name, "conversation") or name == "session.id"`, read 09/28/2026 (control: the same filter returned `messaging.message.conversation_id`).

2. Workflow Tutorial: Optimize DQL Cost

Pattern: schedule a workflow that periodically analyzes DQL execution against ingestion / scan budgets, surfaces high-cost queries, and generates a summary recommending optimization.

When to use: any tenant with material DQL spend — typically OpenPipeline-heavy deployments, ad-hoc analytics workloads, dashboard-heavy environments.

The shape:
1. **Trigger** — scheduled (e.g., weekly) or threshold-based (e.g., daily DQL spend > X)
2. **Analyze** — DQL query against `dt.system.events` and the DPS metering streams to identify highest-cost queries
3. **Summarize** — Generative AI task composes a recommendation
4. **Notify / file** — workflow notification or ticket

Below is a starter query for the analyze step. Tune `from:` and the cost threshold for your tenant.

```
// Top DQL users by scanned bytes — last 7 days
// (Use as the analyze step in a cost-optimization workflow.)
fetch dt.system.query_executions, from:-7d
| filter status == "SUCCEEDED"
| summarize {
    total_scanned_bytes = sum(scanned_bytes),
    execution_count     = count(),
    avg_duration_ms     = avg(execution_duration_ms)
  },
  by:{user.email}
| sort total_scanned_bytes desc
| limit 25
```

Drop `user.email` from the `by:` clause and add `query_string` (truncated) to surface the biggest individual queries. Then pass the top queries to a Dynatrace Intelligence generative AI task in the same workflow for a plain-English explanation and optimization suggestion — the pattern in the *Optimize DQL cost with Workflows* tutorial. (The MCP `explain-dql` tool does the same job for an external agent; a workflow cannot call MCP tools.)

> **Sources:** Optimize DQL cost with Workflows (DT docs) — tutorial built on Dynatrace Intelligence (Preview).

3. Workflow Tutorial: Summarize Open Problems

Pattern: scheduled workflow runs every shift / every morning, gathers active problems, sends to a Generative AI task for narrative summary, and posts to Slack / Teams / email.

When to use: ops teams running shift handoffs; leadership wanting a daily problem digest without staring at the Problems app.

The shape:

1. **Trigger** — cron (e.g., 8 AM weekdays)
2. **Fetch** — query active problems
3. **Summarize** — Generative AI task composes the digest
4. **Post** — Slack / Teams / email notification

> **Sources:** Summarize open problems with Workflows (DT docs) — tutorial built on Dynatrace Intelligence (Preview).

```
// Active and recently-closed problems for a daily digest
fetch dt.davis.problems, from:-24h
| fields display_id, event.name, event.category, event.status,
         event.start, event.end, root_cause_entity_name
| sort event.start desc
| limit 100
```

4. Workflow Tutorial: Forecast Resource Utilization

Pattern: scheduled workflow forecasts capacity-relevant series (host disk, namespace CPU, ingestion volume) using the workflow **Analyze data** action with the *Generic Forecast Analysis* analyzer, and notifies when projected exhaustion hits the threshold.

When to use: capacity planning. Catching disk-full / quota-exhaust scenarios before they fire is the canonical forecast use case.

The shape:
1. **Trigger** — scheduled (e.g., daily)
2. **Query** — capacity series for the relevant resource
3. **Forecast** — analyzer task with confidence interval
4. **Branch** — if projected exhaustion within N days → escalate
5. **Notify** — workflow notification

The tutorial is explicit that this is a workflow action, not an MCP call: *"To trigger the forecast from a workflow, you need the Analyze data action."* External agents reach the same forecasting capability through the MCP `timeseries-forecast` tool (§5).

> **Sources:** Forecast disk and resource utilization (DT docs) — tutorial built on Dynatrace Intelligence (Preview).

```
// Disk usage timeseries — input to the forecast analyzer
// Per host, hourly, last 30 days
timeseries disk_used = avg(dt.host.disk.used.percent),
  by:{dt.entity.host, dt.entity.disk},
  from:-30d,
  interval:1h
| limit 100
```

5. The Dynatrace MCP Server

The MCP server is how your agent (Claude Code, Cursor, GitHub Copilot, etc.) calls Dynatrace as a tool. From a single chat prompt, your agent can generate DQL, execute it, and call analyzers.

Tool families exposed:

Family	Tools
DQL	`create-dql`, `execute-dql`, `explain-dql`
Davis analyzers	`static-threshold-analyzer`, `seasonal-baseline-anomaly-detector`, `adaptive-anomaly-detector`, `timeseries-novelty-detection`, `timeseries-forecast`, `log-pattern-extractor`
Discovery	`find-documents`, `find-troubleshooting-guides`, `ask-dynatrace-docs`
Topology	`get-entity-id`, `get-entity-name`
Davis problems & security	`query-problems`, `get-problem-by-id`, `get-events-for-kubernetes-cluster`, `get-dynatrace-vulnerabilities`, `get-dynatrace-compliance-findings`, `get-security-events-summary`, `get-security-event-details`

Tool names as registered by the Dynatrace MCP gateway, 09/24/2026 — the catalog changes; list your server's tools before scripting against a name.

Setup outline:
1. Create a Platform Token with `mcp-gateway:servers:invoke` and `mcp-gateway:servers:read`, plus the data scopes the tools need (`davis:analyzers:execute`, `storage:events:read`, …)
2. Add the Dynatrace MCP server config to your agent (Claude Code: `.mcp.json` at the

project root, or `claude mcp add` (stored in `~/.claude.json`); Cursor: settings.json; GitHub Copilot: similar)
3. Test with a tool call from the agent

Use cases:
- *In-IDE notebook authoring* — generate DQL while writing the markdown around it
- *Incident response* — Claude Code investigates a deployed service, calls `query-problems` and `execute-dql`, summarizes findings
- *Custom agents* — your own LangChain / LangGraph agent uses Dynatrace MCP as one of many tools

Auth and IAM bind the MCP integration to your IAM policy — same `davis:analyzers:execute` scope as the Anomaly Detection app, same `storage:events:read` for problem queries.

> **Custom alerts move from the app to Settings (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and it is the app the scope comparison above refers to.
>
> **Sources:**
> - Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*
> - Dynatrace MCP server (DT docs) — *"both the user and the token must have the following permissions to access and invoke server tools"* (`mcp-gateway:servers:invoke`, `mcp-gateway:servers:read`)
> - IAM policy statements (DT docs)
> - Connect Claude Code to tools via MCP (Anthropic docs) — *"Project-scoped servers enable team collaboration by storing configurations in a `.mcp.json` file at your project's root directory."*

6. Agentic Patterns: AutomationEngine and Approval-Based Remediation

Agentic workflows are an emerging area in 2026. The intent: detect a problem (Causal AI), propose a remediation (Generative AI), execute under policy guardrails (workflow + AutomationEngine), surface the human approval / post-action audit.

> **Two platform moves in this space (SaaS 1.344 and 1.346).** An **SRE Agent workflow template** ships with 1.344 (rollout from 07/29/2026): production problems trigger automated root-cause analysis and impact annotations. It is a worked instance of exactly the pattern below — detect, analyse, annotate — and lands on the *suggest* side of the line, annotating rather than acting, which is why it is a reasonable first agentic workflow to adopt. Separately, **Dynatrace Assist gains an agentic mode** in 1.346 (rollout from 08/25/2026), where embedded conversation starters perform live environment analysis rather than answering from documentation; 1.344 added role, expertise-level, language, and tone personalisation. Assist moving from answering to analysing puts it inside the guardrail model below — the Scope, Policy and Audit questions now apply to it, not just to workflows you author. The suggest-mode posture below is unchanged by either.
>
> Sources: What's new in Dynatrace SaaS 1.344 (DT docs), What's new in Dynatrace SaaS 1.346 (DT docs)

Three guardrails to enforce:

1. **Scope** — which environments / namespaces / clusters can the agent act on?
2. **Policy** — what action types are allowed without human approval (read-only / limited write / restart) vs. require approval (config change, scaling)?
3. **Audit** — where is the agent's reasoning logged? Audit must capture the prompt, the data the agent saw, and the action taken.

Practical 2026 posture: keep agentic remediation in **suggest mode** for the first 60–90 days. The agent posts the proposed remediation as a Slack / Teams / ticket comment — a human approves, a workflow executes. Move to fully automated only once the suggested-action quality is consistently high in your environment.

7. Cross-Series Pointers

- **WFLOW** — workflows fundamentals (this notebook adds the AI-task lens)
- **AUTOM** — config-as-code, deployment automation
- **AIOPS-04** — Dynatrace Assist as the chat surface that calls many of these same APIs
- **AIOPS-07** — end-to-end Detect → Investigate → Remediate use cases that compose all of the above

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.